

JAVA DOCS

Exception Handling in Java

try-catch-finally, Custom Exceptions, and Best Practices

Java Agentic AI — Knowledge Base Reference

1. The Exception Hierarchy

All exceptions and errors in Java derive from `java.lang.Throwable`, which branches into two subtypes: **Error** (serious JVM-level problems, not meant to be caught — `OutOfMemoryError`, `StackOverflowError`) and **Exception** (application-level problems that code can reasonably handle).

Category	Examples	Must be declared/caught
Checked Exception	<code>IOException</code> , <code>SQLException</code> , <code>ClassNotFoundException</code>	Yes — compiler enforced
Unchecked Exception (<code>RuntimeException</code>)	<code>NullPointerException</code> , <code>ArrayIndexOutOfBoundsException</code> , <code>IllegalArgumentException</code>	No — optional
Error	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>	No — not meant to be handled

Note: Checked exceptions represent recoverable conditions external to the program's control (a missing file, a network failure) that the compiler forces you to acknowledge. Unchecked exceptions usually signal programming bugs (null dereference, bad index, invalid argument).

2. try-catch-finally

```
try {
    int[] arr = new int[5];
    arr[10] = 1; // throws ArrayIndexOutOfBoundsException
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index error: " + e.getMessage());
} catch (Exception e) { // broader catch, must come after specific ones
    System.out.println("General error: " + e);
} finally {
    System.out.println("Always runs - cleanup code here");
}
```

- finally executes whether or not an exception occurred, and even if the try/catch block returns or throws — except if the JVM exits (`System.exit()`) or crashes.
- Catch blocks are evaluated top-to-bottom; a subclass exception type must be caught before its superclass, or the compiler rejects it as unreachable code.
- Multi-catch: `catch (IOException | SQLException e) { ... }` handles multiple unrelated types identically.

3. try-with-resources

Introduced in Java 7, this ensures any resource implementing `AutoCloseable` is closed automatically, in reverse order of declaration, even if an exception is thrown — eliminating verbose manual finally-block cleanup.

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"));
    FileWriter fw = new FileWriter("out.txt")) {
    String line = br.readLine();
    fw.write(line);
} catch (IOException e) {
    e.printStackTrace();
}
// br.close() and fw.close() are called automatically, fw first, then br
```

4. Throwing Exceptions

```
public void withdraw(double amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
    if (amount > balance) {
        throw new InsufficientFundsException("Balance too low");
    }
}

public void readFile(String path) throws IOException { // declares a checked exception
    Files.readAllLines(Paths.get(path));
}
```

5. Custom Exceptions

```
// Checked custom exception
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
    public InsufficientFundsException(String message, Throwable cause) {
        super(message, cause);
    }
}

// Unchecked custom exception
public class InvalidOrderException extends RuntimeException {
    public InvalidOrderException(String message) {
        super(message);
    }
}
```

- Extend `Exception` for a checked custom exception (caller must handle or declare it).
- Extend `RuntimeException` for an unchecked one (common in modern APIs, e.g. Spring's exception model).
- Always provide a constructor that accepts a message and, ideally, a `Throwable` cause to preserve the original stack trace when wrapping a lower-level exception.

6. Exception Chaining

```
try {
    parseConfig();
} catch (ParseException e) {
    throw new ConfigurationException("Failed to load config", e); // 'e' becomes the cause
}

// Later, e.getCause() retrieves the original ParseException, preserving full diagnostic history
```

7. finally, return, and Control Flow Gotchas

```
public int tricky() {
    try {
        return 1;
    } finally {
        System.out.println("finally runs even though try returned");
        // return 2; // DANGER: this would silently discard the try's return value!
    }
}
// tricky() returns 1. If finally also has 'return 2', it returns 2 instead, swallowing
// the original return AND any exception that might have been thrown from the try block.
```

Note: *Never put a return, break, or continue inside a finally block — it silently swallows any exception in flight from the try/catch, which is a subtle and dangerous bug.*

8. Best Practices

- Catch the most specific exception type possible; avoid catching generic Exception or Throwable except at top-level boundary handlers (e.g. a REST controller's global exception handler).
- Never leave a catch block empty ("swallowing" exceptions) — at minimum, log it.
- Don't use exceptions for normal control flow — they're relatively expensive (stack trace capture) and reduce readability.
- Include the original cause when wrapping exceptions, so the root failure is never lost.
- Fail fast with meaningful messages: include the invalid value or relevant context in the message.
- Close resources deterministically with try-with-resources rather than manual finally blocks.

9. Common Runtime Exceptions Reference

Exception	Typical Cause
NullPointerException	Calling a method / accessing a field on a null reference
ArrayIndexOutOfBoundsException	Accessing an array index outside [0, length-1]
ClassCastException	Invalid downcast between incompatible types
NumberFormatException	Parsing an invalid numeric string, e.g. Integer.parseInt("abc")
ArithmeticException	Integer division by zero
ConcurrentModificationException	Structural modification during iteration without using the iterator
IllegalStateException	Method called when the object is in an inappropriate state
UnsupportedOperationException	Calling a mutator on an immutable/fixed-size collection

10. Quick Interview Recap

- Checked vs unchecked? Checked exceptions are verified at compile time and must be declared or caught; unchecked (`RuntimeException` and its subclasses) are not, typically representing bugs.
- Can finally be skipped? Yes — if the JVM is terminated via `System.exit()`, or the JVM crashes, or the thread is killed.
- What's the difference between `throw` and `throws`? `throw` actually raises an exception instance; `throws` is a method-signature declaration listing checked exceptions the method might propagate.
- Can you catch an `Error`? Technically yes (`Error` extends `Throwable`) but it's discouraged — `Errors` usually indicate unrecoverable JVM-level problems.
- Why avoid exceptions for control flow? Constructing an exception captures a full stack trace, which is comparatively expensive, and it obscures normal program logic.